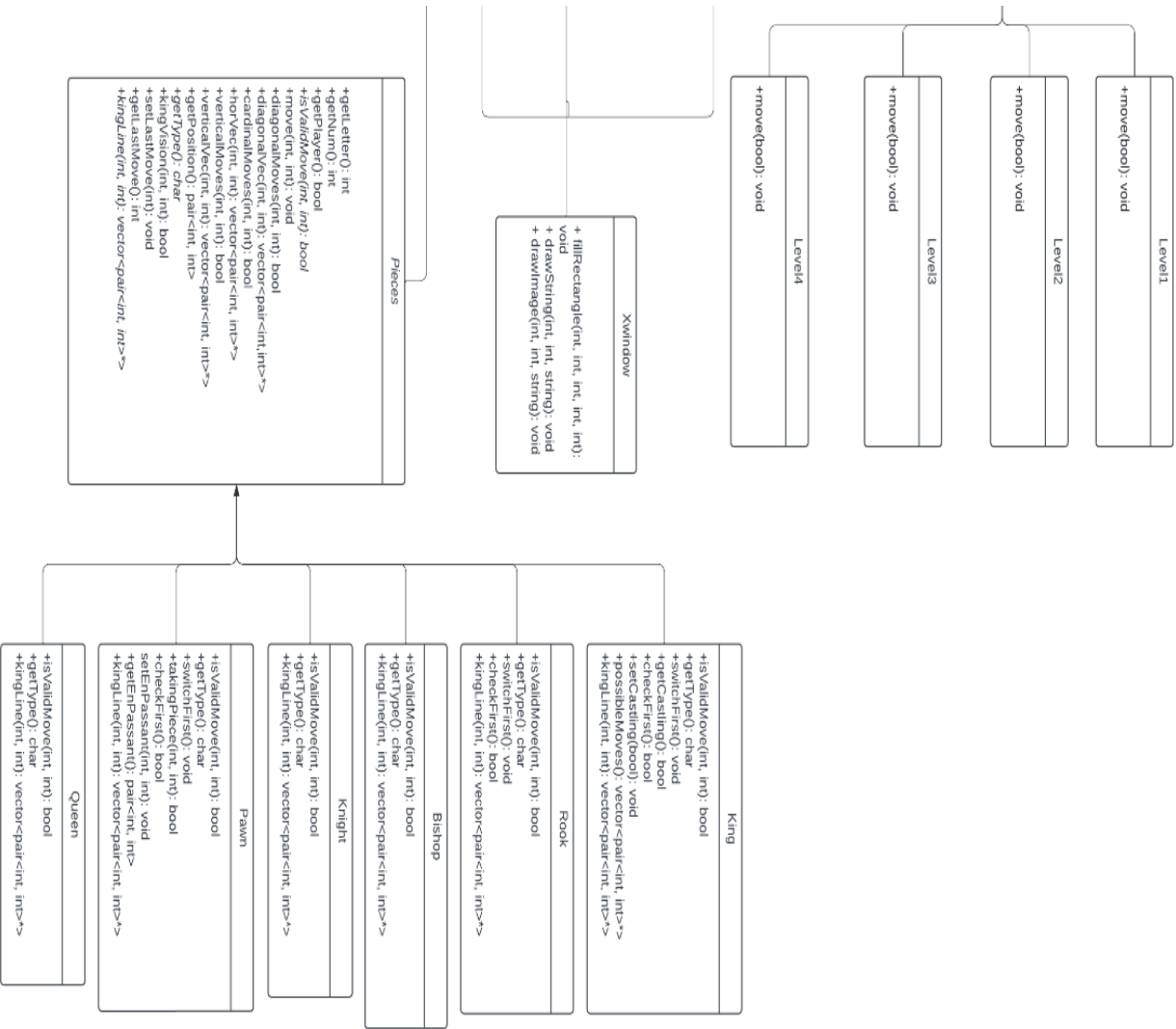


R.Kumar

[illegible]



Overview

As planned in the original UML, we employed the MVC (Model-View-Controller) architecture. There were two major differences between the original UML on due date 1 and the current UML. Firstly, we realized there was no need for **Human** and **AI** classes to inherit from **Player**. **Player** is able to store all the vital information on its own and eliminates the need for subclasses. Another reason for this change is due to the fact that **Controller** needs to interact with the **AI** class directly to pass on moves to **Model**. For these reasons, it is simpler for **Controller** to have an instance of **AI** instead of going through the **Player** class. Secondly, we also had aggregation relationships between certain classes that should have been composition. For instance, in the updated UML, **Model** “owns-a” **Player** and a **Board**, and **Board** “owns-a” **PieceSet**. The reason for this is because we don’t want a **Player** or **Board** to exist outside of **Model**. There is no need to have a **Board** or **Player** class as a standalone class. Similar arguments can be made regarding **Board** and **PieceSet**. Aside from those two major changes, we continued with the design of the MVC architecture. The MVC model allowed us to separate our game into 3 distinct components that fostered few dependencies between modules and permitted changes to the game without multiple implementations. Thus, our main three classes were **Model**, **View**, and **Controller**.

The **Model** class is the centerpiece of chess, and keeps track of all the data flow and logic surrounding the implementation of the game. **Model** interacts with the following classes pertaining to the logic of the game:

- **Player**
- **Board**
- **PieceSet**
- **Pieces** with subclasses **King**, **Rook**, **Bishop**, **Knight**, **Pawn**, and **Queen**

The **View** class employs the Observer Design Pattern where **Model** acts as a concrete subject and **Text** and **Graphics** act as the concrete observers. This allows for a smooth implementation of both textual and graphical displays.

The **Controller** class manages input, and control flow between **Model** and **View**. **Controller** also interacts with **AI** with subclasses **Level1**, **Level2**, **Level3**, **Level4** for computer generated moves.

Design

Model

- **Player**, **Board**, **PieceSet**, **Pieces**, **King**, **Rook**, **Bishop**, **Knight**, **Pawn**, and **Queen**

As mentioned previously, The **Model** class is the centerpiece of chess, and keeps track of all the data flow and logic surrounding the implementation of the game. This class is responsible for interacting with the **Controller** class that provides user input and updating the text and graphical interfaces implemented with the Observer Design Pattern later discussed in the **View** class. Aside from facilitating the information flow between **View** and **Controller**, **Model** also “owns-a” **Player** class and a **Board** class that holds key information regarding the game state. The **Player** class stored in **Model** holds vital information such as the score, and the difficulty of the AI (Level 1, Level 2, Level 3, Level 4) if the player is not human. We will discuss later that the information stored inside **Player** is crucial for the **Controller** class to call an AI move if both players are not human. In order for **Model** to initiate a move, it must act through **Board**. Our **Board** class was designed to represent the physical board in a chess game, serving primarily to validate moves made by players or AI and to provide available moves. The central functionality lies in the `isValidMove()` method, where a move and a piece are examined to determine their validity. Additionally, **Board** incorporates check and checkmate functions, as it possesses the most information about the current state of the board. The implementation of castling and en passant are also included in this class due to its comprehensive understanding of the board's status. The way **Board** represents the chess board is through the class **PieceSet**. **Board** “owns-a” **PieceSet**, which is essentially a vector of 16 **Piece***, each representing the pieces on one side. For this reason, **Board** owns one white and one black **PieceSet**. Each **Piece*** in **PieceSet** points to an instance of 8 **Pawns**, 2 **Rooks**, 2 **Bishops**, 2 **Knights**, 1 **Queen**, and 1 **King**. Each **Piece** provides similar basic functionality when it comes to returning and updating coordinates. However, the behaviour of the inherited pieces from **Pieces** differ when it comes to methods such as `isValidMove()`, `kingLine()` and `kingVision()`. Virtual methods allow the inherited **Pieces** to perform different checks for valid moves and proper insight on the location of the enemy king using `kingLine` and `kingVision`.

View

→ Text and Graphics

As planned in the original UML, we implemented an observer design pattern using **Model** as a concrete subject and **Text** and **Graphics** act as the concrete observers. **Model** “has-a” vector of **View*** that uses `attach(View*)` and `detach(View*)` to attach and detach different **View*** that will observe **Model**. In this case, we had two **View** objects attached to **Model**; being **Text** and **Graphics** as we had one text interface and one graphical interface. When it was time to display the board, **Model** will call `notifyObservers()` that will call `notify()` on each of the attached **View*** in **Model** that will display the board. The method `notify()` will call `model->getState(int, int)` to determine which piece is present at the given coordinate and display the board. **Text** will use `std::cout` to print the text display and **Graphics** will use the **Xwindow** class to display a graphical interface of the chess board.

Controller

→ AI, Level1, Level2, Level3, Level4

The **Controller** class was the class that communicated directly with **main.cc** and **Model** which facilitated the user input with the movement of pieces on the board. The **Controller** class had methods that supported the core commands that would be called. The commands supported by the command interpreter are passed as strings into the **Controller** class to access features such as move, setup commands, and resign. The **Controller** class then communicates appropriately with the **Model** class to issue commands into the actual game. The **Controller** class also “has-a” abstract **AI** class with subclasses **Level1**, **Level2**, **Level3**, and **Level4**, each representing the difficulty of the computer generated moves. If the command game “computer[1]”, “computer[2]”, “computer[3]”, or “computer[4]” is passed into the program, the **Controller** class will call the corresponding move method in the **Level** class to move a piece on the board. Currently, we have implemented Level 1, which permits random legal moves. **Level1** randomly chooses a friendly piece on the board and generates a random legal move that can be performed. Due to time constraints, we were not able to fully implement the other levels, but given the design of our program, the implementation of other levels would integrate seamlessly into our game.

Resilience to Change

Our game code demonstrates high adaptability to various features, making it easy to incorporate additional elements such as additional pieces and piece types, a timer, or the option to play silent chess. When introducing new pieces to the game, the process involves creating a new piece that inherits from the Piece class. Adjustments are then made to accommodate for the new piece’s behaviour in `isValidMove()` and `kingLine()` for instance. `PieceSet` is also expanded to accommodate the new piece. This would allow users to play with an additional piece type and define the rules to how that piece may move. Another feature we could implement into our game fairly easily is having a timer, this would be added in the Controller of the game and we would need to modify the Player file for this implementation so that it will be able to store if a Player has gone overtime or not. It would allow users to play their moves in under a specific amount of time, making the game a little more challenging! Lastly to implement silent chess we would modify the View and Model such that the moves each player makes are not reflected visually to the other person. Furthermore we would have to adapt the Board as we do not have to check for check() and simply must check when the king is caught.

Answers to Questions

Q1: Chess programs usually come with a book of standard opening move sequences, which list accepted opening moves and responses to opponents’ moves, for the first dozen or so moves

of the game. Although you are not required to support this, discuss how you would implement a book of standard openings if required.

Implementing a book of standard opening move sequences would be best done through the use of a dictionary structure since standard opening move sequences have names associated with them, such as “King’s Gambit”, “Queen’s Gambit”, “Dutch Defense”, “London System”, etc. The keys would be strings indicating the names of standard opening move sequences and the values would be a vector of vectors each containing the piece name and the targeted positioning on the board. We would then feed this information to the controller to initialize the game as seen fit by the various opening move sequences.

Q2: *How would you implement a feature that would allow a player to undo their last move? What about an unlimited number of undos?*

To allow users to undo their last move, we would create a vector that maintains a history of game states, storing pointers to each move done. Each time a move is made, the current board state is saved and it would be added to the back of this vector similar to a stack. The board class contains one white and one black PieceSet representing the positions of the pieces on the board. To undo we would have to pop back the last board state in the vector and notify our text and graphical displays accordingly, and would also need to check which color the moves are being undone for. To ensure the undo moves are legal (i.e. the board is not empty and moves can be undone) we would need to check isValidMove() and also make sure the board is not undoing what was done in setup mode.

Q3: *Variations on chess abound. For example, four-handed chess is a variant that is played by four players (search for it!). Outline the changes that would be necessary to make your program into a four-handed chess game.*

One of the biggest changes in four-handed chess from typical two-player chess is the layout of the board which consists of additional 3×8 squares on each of the four sides of the original 8×8 board. This implies the inclusion of an additional $3 \times 8 \times 4 = 24 \times 4 = 96$ squares. Accommodating for this change would require a larger coordinate system contained in each individual piece.

Another significant deviation from the standard game of chess is the number of pieces on the board which goes up from 32 (16 Pawn objects, 4 Rook objects, 4 Knight objects, 4 Bishop objects, 2 Queen objects, and 2 King objects) to 64 (32 Pawn objects, 8 Rook objects, 8 Knight objects, 8 Bishop objects, 4 Queen objects, and 4 King objects). Accommodating for this change would require the additional 32 Piece objects to be added to PieceSet upon initialization.

The addition of two new players, introduces the need for two new colours. As we currently denote a piece’s color using a bool type, we would need to modify the interface to instead use an enum type where each color can be represented by an int type.

Four-handed chess has two main game modes – free-for-all and teams. The main addition necessary in the implementation of the free-for-all is the introduction of a points system, which unlike in a typical chess game, is the determinant of the winner. The teams game has similar features to the standard chess game but is played with two two-player teams. The main modification would be the introduction of a “friendly pieces” concept since members of the same team cannot capture each other’s pieces.

Final Questions

Q1: What lessons did this project teach you about developing software in teams? If you worked alone, what lessons did you learn about writing large programs?

Working as a group made us better understand software development in teams as we had to brainstorm how to approach making the Chess game ourselves. It pushed us to think as a group, debug as a group, and essentially bring all the moving parts together. This was different from all the other programming we have done before because we really had to communicate with each other effectively and express the problems we were having clearly, otherwise it led to a lot of confusion. It was also different because working on such a large project as a team leads to many different components and code that we are not used to (as we didn’t write it someone else did), this was new and took some time to adapt to. There was a lot of trial and error to see if our components of code worked together, and when they didn’t we had to debug it together.

The MVC model allowed us to cleanly separate our game into three distinct components: the model, the view, and the controller. This separation minimized dependencies between modules and allowed us to make changes to the game without affecting everything else. Each module had a specific purpose, allowing clean communication among them and making our code more modular and maintainable.

As a group, we learned valuable lessons about collaboration and version control through using Git. We realized the importance of setting clear conventions and communication channels when working on the same codebase simultaneously, we would make comments when we would push code so that everyone else knew what the new code was for. Utilizing branches, pull requests, and reviewing changes in code became essential for maintaining code quality and preventing conflicts. Overall, this project taught us the significance of teamwork, effective communication, and proper version control practices in software development. Making the Chess game also reinforced the importance of breaking down large programs into manageable components to simplify debugging.

Q2: What would you have done differently if you had the chance to start over?

If given the chance to start over we would begin the planning phase sooner than we did. This would give us more time to better understand what functions we would have in each class. As you can see from our final UML, we have a lot more functions in each class. If we began the planning phase sooner, we would’ve been able to plan for more of the functions rather than

creating them on the fly. Also, we would've been able to better understand the structure of our program. For example, the relationships between the classes, like inheritance or aggregation.

Another thing we could've improved on was getting setup mode working earlier, this would've made testing much easier for smaller features. For example, testing our implementation for en passant, castling and pawn promotion was a tedious process. We had to find ways to put the initial board in a configuration in which those features could be tested. If we had setup mode working sooner, we would've been able to design a board that allowed us to test the features right away. This also applies to testing check and checkmate, testing different scenarios that put a player in check but not checkmate, and other checkmate scenarios.

Lastly, we would also allocate time better to be able to implement some features that we weren't able to add like more AI levels and stalemate. Better time management would've helped us to conserve more time to allocate to these features, potentially earning us a higher grade.